

PARTE 2. COMPRENSIÓN Y PREPARACIÓN DE LOS DATOS

Equipo 6

Importación y estructuración de los datos

Antes de iniciar con la exploración de los datos, es necesario importar y estructurar los datos de los archivos Excel que contienen la información de cada año, ya que se encontró que los archivos contienen varias hojas y que la cantidad de valores nulos varía entre los años. Por lo tanto, se concatenarán las hojas de cada año y se agregará una columna indicativa a la estación de la observación, después se realizará un análisis comparativo de los valores nulos por año para determinar cuál año tiene la menor cantidad de datos faltantes y, por ende, será el año seleccionado para la exploración de los datos.

```
import pandas as pd
import numpy as np

# Lista para guardar el resumen de calidad de cada año
resultados = []

# Ciclo para procesar los archivos Excel del 2020 al 2025
for anio in range(2020, 2026):
    nombre_excel = f'BD {anio}.xlsx'
    nombre_csv_salida = f'BD_{anio}_Completo.csv'

    try:
        diccionario_hojas = pd.read_excel(nombre_excel, sheet_name=None)
        lista_tablas = []

        for nombre_pestana, df_temporal in diccionario_hojas.items():
            df_temporal['Estacion'] = nombre_pestana
            lista_tablas.append(df_temporal)
```

```

df_anio_completo = pd.concat(lista_tablas, ignore_index=True)
df_anio_completo.to_csv(nombre_csv_salida, index=False)

total_filas = len(df_anio_completo)
total_nulos = df_anio_completo.isna().sum().sum()

resultados.append({
    'Año': anio,
    'Total Filas': total_filas,
    'Valores Nulos': total_nulos,
    'Archivo Generado': nombre_csv_salida
})

except FileNotFoundError:
    pass

# Mostrar la tabla comparativa ordenada por cantidad de nulos
if resultados:
    df_comparativo = pd.DataFrame(resultados)
    print("\n--- Comparativa de Calidad de Datos por Año ---")
    print(df_comparativo.sort_values(by='Valores Nulos'))

```

```

--- Comparativa de Calidad de Datos por Año ---
   Año  Total Filas  Valores Nulos  Archivo Generado
2  2022      123383      84056  BD_2022_Completo.csv
3  2023      131370     103402  BD_2023_Completo.csv
4  2024      131741     139587  BD_2024_Completo.csv
1  2021      122629     197665  BD_2021_Completo.csv
5  2025      131378     212002  BD_2025_Completo.csv
0  2020      114102     552931  BD_2020_Completo.csv

```

Como podemos observar, el año 2022 es el que tiene la menor cantidad de valores nulos, por lo que se seleccionará este año para la exploración de los datos.

Comprensión de los datos

Una vez seleccionado el año 2022 por poseer la mejor integridad en sus registros, procedemos a realizar la exploración de su estructura antes de cualquier modificación.

```
# Cargar la base de datos consolidada del 2022
df_2022 = pd.read_csv('BD_2022_Completo.csv')

# a) Dimensión del dataset
print(f"Dimensión del dataset: {df_2022.shape[0]} registros y {df_2022.shape[1]} columnas.\n")

# b) Información general y tipos de datos
print("--- Información de las variables ---")
df_2022.info()

# c) Calidad de los datos: Valores nulos y duplicados
print("\n--- Porcentaje de Valores Nulos por Variable ---")
nulos_pct = (df_2022.isna().sum() / len(df_2022)) * 100
print(nulos_pct.round(2).astype(str) + ' %')

print(f"\nCantidad de registros duplicados: {df_2022.duplicated().sum()}")
```

Dimensión del dataset: 123383 registros y 17 columnas.

```
--- Información de las variables ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123383 entries, 0 to 123382
Data columns (total 17 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Fecha y hora    123383 non-null object
1   CO              119121 non-null float64
2   NO              118291 non-null float64
3   NO2             119035 non-null float64
4   NOX             119060 non-null float64
5   O3              118878 non-null float64
6   PM10            119447 non-null float64
7   PM2.5           102325 non-null float64
8   PRS             120054 non-null float64
9   RAINF           121332 non-null float64
10  RH              118631 non-null float64
11  SO2             112465 non-null float64
12  SR              119487 non-null float64
13  TOUT            120775 non-null float64
14  WSR             120157 non-null float64
15  WDR             117631 non-null float64
16  Estacion        123383 non-null object
```

```
dtypes: float64(15), object(2)
memory usage: 16.0+ MB
```

```
--- Porcentaje de Valores Nulos por Variable ---
```

Fecha y hora	0.0 %
CO	3.45 %
NO	4.13 %
NO2	3.52 %
NOX	3.5 %
O3	3.65 %
PM10	3.19 %
PM2.5	17.07 %
PRS	2.7 %
RAINF	1.66 %
RH	3.85 %
SO2	8.85 %
SR	3.16 %
TOUT	2.11 %
WSR	2.61 %
WDR	4.66 %
Estacion	0.0 %

```
dtype: object
```

Cantidad de registros duplicados: 0

Descripción de las variables involucradas

- Fecha y hora: (Temporal/Datetime) Indica el momento exacto de la captura de los datos.
- Estacion: (Categorica) Ubicación del sensor de monitoreo. Variable nominal.
- CO, NO, NO2, NOX, O3, SO2: (Numéricas continuas) Concentración de gases contaminantes en partes por billón/millón.
- PM10, PM2.5: (Numéricas continuas) Material particulado en suspensión ($\mu\text{g}/\text{m}^3$). PM10 será nuestra variable objetivo.
- PRS, RH, SR, TOUT, WSR, WDR: (Numéricas continuas) Variables meteorológicas: Presión (mmHg), Humedad Relativa (%), Radiación Solar (W/m^2), Temperatura ($^{\circ}\text{C}$), Velocidad (km/h) y Dirección del viento.
- RAINF: (Numérica) Precipitación. Se identificó como un valor espurio, ya que consta exclusivamente de valores 0.0 en casi todos sus registros.

Preparación de los datos

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# ---> AQUÍ ESTÁ LA CORRECCIÓN <---
# Primero, debemos crear df_prep a partir de df_2022 (y de paso eliminamos duplicados)
df_prep = df_2022.drop_duplicates().copy()

# Corregir valores espurios: Eliminar variable RAINF por carecer de varianza
df_prep = df_prep.drop(columns=['RAINF'])

# Tratamiento de Fechas y preparación para imputar
df_prep['Fecha y hora'] = pd.to_datetime(df_prep['Fecha y hora'])
df_prep['Mes'] = df_prep['Fecha y hora'].dt.month
df_prep['Hora'] = df_prep['Fecha y hora'].dt.hour

# Extraemos la fecha temporalmente ya que el imputador no acepta Datetime
fechas = df_prep['Fecha y hora']
df_prep = df_prep.drop(columns=['Fecha y hora'])

# Manejo de Datos Categóricos (Variables Dummy)
df_prep = pd.get_dummies(df_prep, columns=['Estacion'], drop_first=True)

# Imputación de datos faltantes (IterativeImputer)
print("Iniciando imputación iterativa (esto puede tomar un momento)...")
imp = IterativeImputer(max_iter=10, random_state=42)

# Guardamos el nombre de las columnas
columnas = df_prep.columns
# Aplicamos imputación a TODO el dataset
df_imputado_array = imp.fit_transform(df_prep)

# Reconstruimos el DataFrame
df_clean = pd.DataFrame(df_imputado_array, columns=columnas)
# Reintegramos la columna de Fecha y hora
df_clean.insert(0, 'Fecha y hora', fechas.values)

print("Imputación finalizada sin nulos restantes")
```

Iniciando imputación iterativa (esto puede tomar un momento)...

Imputación finalizada sin nulos restantes

Manejo de Valores Atípicos (Outliers)

```
# Variables numéricas a tratar por outliers
cols_outliers = ['PM10', 'PM2.5', 'O3', 'CO']

for col in cols_outliers:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    limite_inf = Q1 - 1.5 * IQR
    limite_sup = Q3 + 1.5 * IQR

    # Aplicar Capping (Clipping): limitar los valores a los rangos calculados
    df_clean[col] = np.where(df_clean[col] > limite_sup, limite_sup, df_clean[col])
    df_clean[col] = np.where(df_clean[col] < limite_inf, limite_inf, df_clean[col])

print("Outliers tratados mediante método de Clipping (IQR).")
```

Outliers tratados mediante método de Clipping (IQR).

Transformación y Reestructuración de Datos

```
from sklearn.preprocessing import StandardScaler

# Discretizar datos (Binning) - Creando categorías de Calidad del Aire para PM10
bins_pm10 = [0, 50, 100, np.inf]
etiquetas = ['Buena', 'Regular', 'Mala']
df_clean['Calidad_Aire_PM10'] = pd.cut(df_clean['PM10'], bins=bins_pm10, labels=etiquetas)

# Escalar y normalizar los datos (StandardScaler)
cols_numericas = ['CO', 'NO', 'NO2', 'NOX', 'O3', 'PM10', 'PM2.5', 'PRS', 'RH', 'SR', 'TOUT']

scaler = StandardScaler()
# Creamos copias escaladas para mantener la legibilidad de la base principal si se desea,
df_clean[cols_numericas] = scaler.fit_transform(df_clean[cols_numericas])

# Guardado de la base final reestructurada
df_clean.to_csv('BD_2022_Limpia_Final.csv', index=False)
print("Base de datos limpia y transformada guardada como 'BD_2022_Limpia_Final.csv'")
```

Base de datos limpia y transformada guardada como 'BD_2022_Limpia_Final.csv'